



PROGRAMA INCIDENCIAS

KHALED, NAYEB KHIL | THIAGO BART, SESSELER | JESÚS, BARRU PETRULE

ÍNDICE

DISEÑO PERCEPTIVO DE REQUISITOS.....	2
Gestión de usuarios:	2
Incidencias:	2
Soluciones y valoración:	2
Comunicación:	2
Logros y recompensas:.....	2
Zonas:.....	2
DISEÑO CONCEPTUAL.....	3
DISEÑO LOGICO	3
DDL DE LA BASE DE DATOS.....	1
DML DE LA BASE DE DATOS.....	5
EXPLICACION CONSULTAS.....	10
VISTAS.....	15
FUNCIONES.....	16
TRIGGERS.....	20
INTEFAZ GRAFICA	23
Ventana Inicio Sesión	24
Ventana Registrarse	25
Ventana Admin.....	26
Ventana Usuario.....	29
Ventana Ver Incidencias	30
Ventana Contactar Admin.....	32
Ventana Ver Perfil.....	32

DISEÑO PERCEPTIVO DE REQUISITOS

El programa permite a los usuarios **reportar** incidencias y **resolver** las que otros han creado. Cualquier usuario puede actuar como **reportador** y como **colaborador**.

Gestión de usuarios:

Registro de usuarios con nombre, email, contraseña y fecha de registro.

Los usuarios pueden indicar categorías en las que están **especializados**, lo que ayuda a mostrarles incidencias relacionadas.

Los usuarios pueden **seguir** a otros, ver sus **logros**, su historial de **incidencias** creadas y sus **soluciones** aportadas.

Incidencias:

Un usuario puede crear incidencias indicando **título, descripción, fecha y zona**.

Cada incidencia puede pertenecer a **una o varias categorías**.

Varias personas pueden proponer soluciones, pero solo una puede ser **aceptada** por el creador.

Soluciones y valoración:

Los colaboradores envían soluciones a incidencias resueltas.

Se pueden añadir **comentarios** a las soluciones.

Cuando una solución es aceptada, el reportador **valora** al colaborador, aumentando su reputación/confianza.

Comunicación:

Los usuarios pueden enviarse **mensajes** entre ellos (contenido, fecha y si ha sido leído).

Logros y recompensas:

Los usuarios pueden **obtener** "logros" por su actividad.

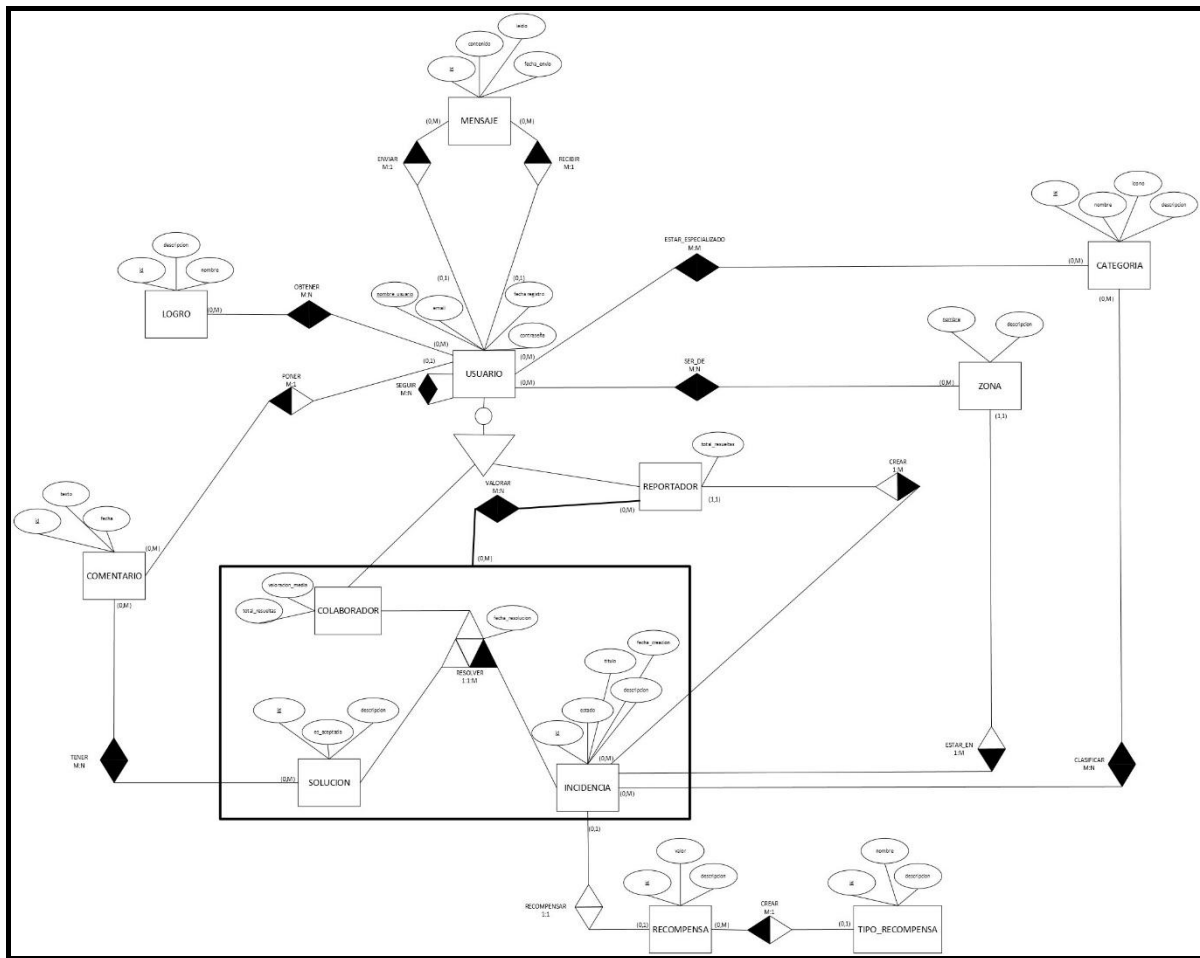
Algunas incidencias incluyen **recompensas**, clasificadas por tipo, y solo puede **asignarse** una recompensa por incidencia.

Zonas:

Las incidencias están **asociadas** a una zona.

Los usuarios pueden **pertenecer** a una o varias zonas para definir su ámbito de actuación.

DISEÑO CONCEPTUAL



DISEÑO LOGICO

USUARIO(nombre_usuario, email,
fecha_registro,contraseña)

PROGRAMA INCIDENCIAS

```

        PK(nombre_usuario)

COLABORADOR(usuario,valoracion_media,total_resu
eltas)

        PK(usuario)
        FK(usuario) -> USUARIO

REPORTADOR(usuario, total_resueltas)

        PK(usuario)
        FK(usuario) -> USUARIO

SEGUIR(usuario1,usuario2)

        PK(usuario1,usuario2)
        FK(usuario1) -> USUARIO
        FK(usuario2) -> USUARIO

MENSAJE(id,contenido,leido,fecha_envio,usuario_

        PK(id)
        FK(usuario_envia) -> USUARIO
        FK(usuario_recibe) -> USUARIO
        LOGRO(id,descripcion,nombre)
        PK(id)

OBTENER(logro,usuario)

        PK(logro,usuario)
        FK(logro) -> LOGRO
        FK(usuario) -> USUARIO

COMENTARIO(id,texto,fecha,usuario)

        PK(id)
        FK(usuario) -> USUARIO

SOLUCION(id,es_aceptada,descripcion)

        PK(id)

TENER(comentario, solucion)

        PK(comentario,solucion)
        FK(comentario) -> COMENTARIO
        FK(solucion) -> SOLUCION

ZONA(nombre,descripcion)

        PK(nombre)

SER_DE(usuario,zona)

        PK(usuario,zona)
        FK(usuario) -> USUARIO
        FK(zona) -> ZONA

INCIDENCIA(id,estado,titulo,descripcion,fecha_c
reacion, reportador, zona)

        PK(id)
        VNN(reportador)
        VNN(zona)
        FK(reportador) -> REPORTADOR
        FK(zona) -> ZONA

RESOLVER(colaborador,incidencia,solucion)

        PK(incidencia,solucion)
        UK(incidencia,colaborador)
        FK(incidencia) -> INCIDENCIA
        FK(solucion) -> SOLUCION
        FK(colaborador) -> COLABORADOR

VALORAR(resolver,reportador,
incidencia,solucion)

        PK(incidencia,solucion,reportador)
        FK(incidencia,solucion) -> RESOLVER
        FK(reportador) -> REPORTADOR

CATEGORIA(id, nombre,icono,descripcion)

        PK(id)

ESTAR_ESPECIALIZADO(usuario,categoria)

        PK(usuario,categoria)
        FK(usuario) -> USUARIO
        FK(categoria) -> CATEGORIA

CLASIFICAR(incidencia,categoria)

        PK(incidencia,categoria)
        FK(incidencia) -> INCIDENCIA
        FK(categoria) -> CATEGORIA

TIPO_RECOMPENSA(id,nombre,descripcion)

        PK(id)

RECOMPENSA(id,valor,descripcion,tipo_recompensa
)

        PK(id)
        FK(tipo_recompensa) -> TIPO_RECOMPENSA

RECOMPENSAR(incidencia,recompensa)

        PK(incidencia)
        UK(recompensa)
        FK(incidencia) -> INCIDENCIA
        FK(recompensa) -> RECOMPENSA
```

DDL DE LA BASE DE DATOS

```
CREATE TABLE USUARIO (
    nombre_usuario VARCHAR(50) PRIMARY KEY,
```

```
email VARCHAR(100),
fecha_registro TIMESTAMP,
contrasena VARCHAR(255)
);
CREATE TABLE ZONA (
    id NUMERIC(10,0) PRIMARY KEY,
    nombre VARCHAR(100)
);
CREATE TABLE CATEGORIA (
    id NUMERIC(10,0) PRIMARY KEY,
    nombre VARCHAR(100),
    descripcion VARCHAR(500)
);
CREATE TABLE TIPO_RECOMPENSA (
    id NUMERIC(10,0) PRIMARY KEY,
    nombre VARCHAR(100),
    descripcion VARCHAR(500)
);
CREATE TABLE LOGRO (
    id NUMERIC(10,0) PRIMARY KEY,
    descripcion VARCHAR(500),
    nombre VARCHAR(100)
);
CREATE TABLE COLABORADOR (
    usuario VARCHAR(50) PRIMARY KEY,
    valoracion_media NUMERIC(3,2),
    total_resueltas NUMERIC(10,0),
    FOREIGN KEY (usuario) REFERENCES USUARIO(nombre_usuario)
);
CREATE TABLE REPORTADOR (
    usuario VARCHAR(50) PRIMARY KEY,
    total_creadas NUMERIC(10,0),
    FOREIGN KEY (usuario) REFERENCES USUARIO(nombre_usuario)
);
```

```
CREATE TABLE SER_DE (  
    usuario VARCHAR(50),  
    id_zona NUMERIC(10,0),  
    PRIMARY KEY (usuario, id_zona),  
    FOREIGN KEY (usuario) REFERENCES USUARIO(nombre_usuario),  
    FOREIGN KEY (id_zona) REFERENCES ZONA(id)  
);  
  
CREATE TABLE ESTAR_ESPECIALIZADO (  
    usuario VARCHAR(50),  
    categoria NUMERIC(10,0),  
    PRIMARY KEY (usuario, categoria),  
    FOREIGN KEY (usuario) REFERENCES USUARIO(nombre_usuario),  
    FOREIGN KEY (categoria) REFERENCES CATEGORIA(id)  
);  
  
CREATE TABLE SEGUIR (  
    usuario1 VARCHAR(50),  
    usuario2 VARCHAR(50),  
    PRIMARY KEY (usuario1, usuario2),  
    FOREIGN KEY (usuario1) REFERENCES USUARIO(nombre_usuario),  
    FOREIGN KEY (usuario2) REFERENCES USUARIO(nombre_usuario)  
);  
  
CREATE TABLE MENSAJE (  
    id NUMERIC(10,0) PRIMARY KEY,  
    contenido VARCHAR(1000),  
    fecha_envio TIMESTAMP,  
    usuario_envia VARCHAR(50),  
    usuario_recibe VARCHAR(50),  
    FOREIGN KEY (usuario_envia) REFERENCES USUARIO(nombre_usuario),  
    FOREIGN KEY (usuario_recibe) REFERENCES USUARIO(nombre_usuario)  
);  
  
CREATE TABLE INCIDENCIA (  
    id NUMERIC(10,0) PRIMARY KEY,  
    estado VARCHAR(50),  
    titulo VARCHAR(200),  
    descripcion VARCHAR(1000),  
    fecha_creacion TIMESTAMP,
```

```
reportador VARCHAR(50) NOT NULL,  
zona NUMERIC(10,0),  
FOREIGN KEY (reportador) REFERENCES REPORTADOR(usuario),  
FOREIGN KEY (zona) REFERENCES ZONA(id)  
);  
  
CREATE TABLE SOLUCION (  
    id NUMERIC(10,0) PRIMARY KEY,  
    es_aceptada BOOLEAN,  
    descripcion VARCHAR(1000)  
);  
  
CREATE TABLE RESOLVER (  
    colaborador VARCHAR(50),  
    incidencia NUMERIC(10,0),  
    solucion NUMERIC(10,0),  
    PRIMARY KEY (incidencia, solucion),  
    UNIQUE (incidencia, colaborador),  
    FOREIGN KEY (incidencia) REFERENCES INCIDENCIA(id),  
    FOREIGN KEY (solucion) REFERENCES SOLUCION(id),  
    FOREIGN KEY (colaborador) REFERENCES COLABORADOR(usuario)  
);  
  
CREATE TABLE VALORAR (  
    incidencia NUMERIC(10,0),  
    solucion NUMERIC(10,0),  
    reportador VARCHAR(50),  
    PRIMARY KEY (incidencia, solucion, reportador),  
    FOREIGN KEY (incidencia, solucion)  
        REFERENCES RESOLVER(incidencia, solucion),  
    FOREIGN KEY (reportador) REFERENCES REPORTADOR(usuario)  
);  
  
CREATE TABLE COMENTARIO (  
    id NUMERIC(10,0) PRIMARY KEY,  
    texto VARCHAR(1000),  
    fecha TIMESTAMP,  
    usuario VARCHAR(50),  
    FOREIGN KEY (usuario) REFERENCES USUARIO(nombre_usuario)  
);
```

```
CREATE TABLE TENER (  
    comentario NUMERIC(10,0),  
    solucion NUMERIC(10,0),  
    PRIMARY KEY (comentario, solucion),  
    FOREIGN KEY (comentario) REFERENCES COMENTARIO(id),  
    FOREIGN KEY (solucion) REFERENCES SOLUCION(id)  
);  
  
CREATE TABLE RECOMPENSA (  
    id NUMERIC(10,0) PRIMARY KEY,  
    valor NUMERIC(10,2),  
    descripcion VARCHAR(500),  
    tipo_recompensa NUMERIC(10,0),  
    FOREIGN KEY (tipo_recompensa) REFERENCES TIPO_RECOMPENSA(id)  
);  
  
CREATE TABLE RECOMPENSAR (  
    incidencia NUMERIC(10,0) PRIMARY KEY,  
    recompensa NUMERIC(10,0) UNIQUE,  
    FOREIGN KEY (incidencia) REFERENCES INCIDENCIA(id),  
    FOREIGN KEY (recompensa) REFERENCES RECOMPENSA(id)  
);  
  
CREATE TABLE CLASIFICAR (  
    incidencia NUMERIC(10,0),  
    categoria NUMERIC(10,0),  
    PRIMARY KEY (incidencia, categoria),  
    FOREIGN KEY (incidencia) REFERENCES INCIDENCIA(id),  
    FOREIGN KEY (categoria) REFERENCES CATEGORIA(id)  
);  
  
CREATE TABLE OBTENER (  
    logro NUMERIC(10,0),  
    usuario VARCHAR(50),  
    PRIMARY KEY (logro, usuario),  
    FOREIGN KEY (logro) REFERENCES LOGRO(id),  
    FOREIGN KEY (usuario) REFERENCES USUARIO(nombre_usuario)  
);
```

DML DE LA BASE DE DATOS

```
-- TABLA USUARIO

INSERT INTO USUARIO (nombre_usuario, email, fecha_registro, contrasena) VALUES

('JesusBP', 'jesbarpet@alu.edu.gva.es', '2026-01-10 10:15:00', 'hash123'), --hash porque se supone que
debemos guardar el hash de la contraseña y no la contraseña en si

('KhaledKN', 'khanay@alu.edu.gva.es', '2026-01-12 12:30:00', 'hash456'),

('ThiagoBS', 'thises@alu.edu.gva.es', '2026-01-15 09:45:00', 'hash789'),

('Usuario01', 'usuario01@alu.edu.gva.es', '2026-01-18 16:20:00', 'hash101'),

('Usuario02', 'usuario02@alu.edu.gva.es', '2026-01-20 18:05:00', 'hash202');


-- TABLA ZONA

INSERT INTO ZONA (id, nombre) VALUES

(1, 'Valencia'),

(2, 'Madrid'),

(3, 'Barcelona'),

(4, 'Sevilla'),

(5, 'Bilbao');


-- TABLA CATEGORIA

INSERT INTO CATEGORIA (id, nombre, descripcion) VALUES

(1, 'Redes', 'Incidencias relacionadas con problemas de red'),

(2, 'Base de datos', 'Incidencias relacionadas con bases de datos'),

(3, 'Frontend', 'Incidencias relacionadas con la interfaz de usuario'),

(4, 'Backend', 'Incidencias relacionadas con la lógica de servidor'),

(5, 'Seguridad', 'Incidencias relacionadas con seguridad');


-- TABLA TIPO_RECOMPENSA

INSERT INTO TIPO_RECOMPENSA (id, nombre, descripcion) VALUES

(1, 'Monetaria', 'Recompensa económica por resolver incidencias'),

(2, 'Puntos', 'Puntos de reputación por contribuciones'),

(3, 'Badge', 'Insignias virtuales por logros'),

(4, 'Gift Card', 'Tarjetas regalo como recompensa'),

(5, 'Certificado', 'Certificados de reconocimiento');


-- TABLA LOGRO
```

```
INSERT INTO LOGRO (id, descripcion, nombre) VALUES
(1, 'Resolver 10 incidencias como colaborador', 'Colaborador Novato'),
(2, 'Reportar 5 incidencias', 'Reportador Inicial'),
(3, 'Recibir valoración perfecta de una solución', 'Valoración Perfecta'),
(4, 'Estar especializado en 3 categorías', 'Especialista'),
(5, 'Resolver 50 incidencias', 'Colaborador Experto');

-- TABLA COLABORADOR
INSERT INTO COLABORADOR (usuario, valoracion_media, total_resueltas) VALUES
('JesusBP', 4.20, 8),
('KhaledKN', 4.90, 6),
('ThiagoBS', 4.50, 9),
('Usuario01', 4.10, 5),
('Usuario02', 3.85, 3);

-- TABLA REPORTADOR
INSERT INTO REPORTADOR (usuario, total_creadas) VALUES
('JesusBP', 8),
('KhaledKN', 5),
('ThiagoBS', 6),
('Usuario01', 3),
('Usuario02', 4);

-- TABLA SER_DE
INSERT INTO SER_DE (usuario, id_zona) VALUES
('JesusBP', 1), -- Valencia
('KhaledKN', 2), -- Madrid
('ThiagoBS', 3), -- Barcelona
('Usuario01', 4), -- Sevilla
('Usuario02', 5); -- Bilbao

-- TABLA ESTAR_ESPECIALIZADO
INSERT INTO ESTAR_ESPECIALIZADO (usuario, categoria) VALUES
```

```

('JesusBP', 1),
('KhaledKN', 2),
('ThiagoBS', 3),
('Usuario01', 4),
('Usuario02', 5);

-- TABLA INCIDENCIA
INSERT INTO INCIDENCIA (id, estado, titulo, descripcion, fecha_creacion, reportador, zona) VALUES
(1, 'Abierta', 'Error al cargar la página', 'La página principal no carga correctamente en ciertos navegadores.', '2026-02-01 09:00:00', 'JesusBP', 1),
(2, 'En progreso', 'Problema con la base de datos', 'Al actualizar los registros, algunos datos no se guardan.', '2026-02-02 10:30:00', 'KhaledKN', 2),
(3, 'Cerrada', 'Permisos incorrectos de usuario', 'Algunos usuarios no pueden acceder a ciertas funciones.', '2026-02-03 11:15:00', 'ThiagoBS', 3),
(4, 'Abierta', 'Consulta lenta en el sistema', 'La consulta SQL tarda demasiado tiempo en ejecutarse.', '2026-02-04 12:45:00', 'Usuario01', 4),
(5, 'En progreso', 'Validación de formularios insuficiente', 'Se pueden enviar datos erróneos sin avisos.', '2026-02-05 14:00:00', 'Usuario02', 5);

-- TABLA CLASIFICAR
INSERT INTO CLASIFICAR (incidencia, categoria) VALUES
(1, 3), -- Error al cargar la página -> Frontend
(2, 2), -- Problema con la base de datos -> Base de datos
(3, 5), -- Permisos incorrectos -> Seguridad
(4, 4), -- Consulta lenta -> Backend
(5, 1); -- Validación insuficiente -> Redes (ejemplo)

-- TABLA RECOMPENSA
INSERT INTO RECOMPENSA (id, valor, descripcion, tipo_recompensa) VALUES
(1, 50.00, '50€ por resolver incidencia crítica', 1),
(2, 100.00, '100 puntos por colaboración destacada', 2),
(3, 0.00, 'Badge de Especialista en Seguridad', 3),
(4, 25.00, 'Gift card Amazon', 4),
(5, 0.00, 'Certificado de participación', 5);

-- TABLA SOLUCION
INSERT INTO SOLUCION (id, es_aceptada, descripcion) VALUES
(1, TRUE, 'Reiniciar el servidor y limpiar la caché.');
```

```

(2, FALSE, 'Actualizar la base de datos a la versión más reciente. '),
(3, TRUE, 'Cambiar la configuración de permisos de usuario. '),
(4, FALSE, 'Optimizar la consulta SQL que genera el error. '),
(5, TRUE, 'Agregar validación de datos en el formulario de entrada. ');

-- TABLA RESOLVER (si tuvieras datos)
-- INSERT INTO RESOLVER (colaborador, incidencia, solucion) VALUES ...;

-- TABLA VALORAR (si tuvieras datos)
-- INSERT INTO VALORAR (incidencia, solucion, reportador) VALUES ...;

-- TABLA COMENTARIO
INSERT INTO COMENTARIO (id, texto, fecha, usuario) VALUES
(1, 'Creo que esta incidencia es urgente.', '2026-02-05 10:00:00', 'JesusBP'),
(2, 'Estoy revisando la incidencia 12.', '2026-02-05 11:15:00', 'KhaledKN'),
(3, 'Necesito más información sobre la zona afectada.', '2026-02-06 09:30:00', 'ThiagoBS'),
(4, 'He completado mi revisión de la incidencia.', '2026-02-06 14:45:00', 'Usuario01'),
(5, 'Gracias por la ayuda, todo solucionado.', '2026-02-07 16:20:00', 'Usuario02');

-- TABLA TENER
INSERT INTO TENER (comentario, solucion) VALUES
(1, 1), -- Comentario de JesusBP pertenece a la solución 1
(2, 2), -- Comentario de KhaledKN pertenece a la solución 2
(3, 3), -- Comentario de ThiagoBS pertenece a la solución 3
(4, 4), -- Comentario de Usuario01 pertenece a la solución 4
(5, 5); -- Comentario de Usuario02 pertenece a la solución 5

-- TABLA SEGUIR
INSERT INTO SEGUIR (usuario1, usuario2) VALUES
('JesusBP', 'KhaledKN'),
('JesusBP', 'ThiagoBS'),

```

```
('KhaledKN', 'JesusBP'),
('ThiagoBS', 'Usuario01'),
('Usuario02', 'JesusBP');

-- TABLA MENSAJE
INSERT INTO MENSAJE (id, contenido, fecha_envio, usuario_envia, usuario_recibe) VALUES
(1, 'Hola Khaled, ¿has visto la incidencia 12?', '2026-02-01 09:10:00', 'JesusBP', 'KhaledKN'),
(2, 'Sí, estoy trabajando en ella', '2026-02-01 09:15:00', 'KhaledKN', 'JesusBP'),
(3, 'Thiago, necesito tu ayuda con la zona 3', '2026-02-02 11:30:00', 'JesusBP', 'ThiagoBS'),
(4, 'Usuario01, revisa tu reporte de ayer', '2026-02-03 14:20:00', 'Usuario02', 'Usuario01'),
(5, 'Perfecto, lo revisaré', '2026-02-03 14:35:00', 'Usuario01', 'Usuario02');

-- TABLA RESOLVER
INSERT INTO RESOLVER (colaborador, incidencia, solucion) VALUES
('JesusBP', 1, 1),
('KhaledKN', 2, 2),
('ThiagoBS', 3, 3),
('Usuario01', 4, 4),
('Usuario02', 5, 5);

-- TABLA RECOMPENSAR
INSERT INTO RECOMPENSAR (incidencia, recompensa) VALUES
(1, 1),
(2, 2),
(3, 3),
(4, 4),
(5, 5);

-- TABLA VALORAR
INSERT INTO VALORAR (incidencia, solucion, reportador) VALUES
(1, 1, 'JesusBP'),
(2, 2, 'KhaledKN'),
(3, 3, 'ThiagoBS'),
```

```
(4, 4, 'Usuario01'),
(5, 5, 'Usuario02');
```

EXPLICACION CONSULTAS

A1: Obtiene el nombre de usuario, email y fecha de registro de todos los usuarios del sistema

```
3 SELECT nombre_usuario, email, fecha_registro
4 FROM USUARIO;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	nombre_usuario [PK] character varying (50)	email character varying (100)	fecha_registro timestamp without time zone
1	JesusBP	jesbarpet@alu.edu.gva...	2026-01-10 10:15:00
2	KhaledKN	khanay@alu.edu.gva.es	2026-01-12 12:30:00
3	ThiagoBS	thises@alu.edu.gva.es	2026-01-15 09:45:00
4	Usuario01	usuario01@alu.edu.gva...	2026-01-18 16:20:00
5	Usuario02	usuario02@alu.edu.gva...	2026-01-20 18:05:00

A2: Obtiene el usuario y la valoración media de los colaboradores con valoración superior a 4.5

```
1 --a.2
2 SELECT usuario, valoracion_media
3 FROM COLABORADOR
4 WHERE valoracion_media > 4.5;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	usuario [PK] character varying (50)	valoracion_media numeric (3,2)
1	KhaledKN	4.90

A3: Obtiene el id, título y fecha de creación de las incidencias que están en estado 'Abierta'

```
1 SELECT id, titulo, fecha_creacion
2 FROM INCIDENCIA
3 WHERE estado = 'Abierta';
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	id [PK] numeric (10)	titulo character varying (200)	fecha_creacion timestamp without time zone
1	1	Error al cargar la página	2026-02-01 09:00:00
2	4	Consulta lenta en el siste...	2026-02-04 12:45:00

A4: Obtiene el id, descripción y valor de todas las recompensas ordenadas de mayor a menor valor

PROGRAMA INCIDENCIAS

```
1 SELECT id, descripcion, valor
2 FROM RECOMPENSA
3 ORDER BY valor DESC;
```

Showing rows: 1 to 5 Page No: 1 of 1		
id [PK] numeric (10)	descripcion character varying (500)	valor numeric (10,2)
2	100 puntos por colaboración destaca...	100.00
1	50€ por resolver incidencia crítica	50.00
4	Gift card Amazon	25.00
3	Badge de Especialista en Seguridad	0.00
5	Certificado de participación	0.00

A5: Obtiene el nombre de usuario y fecha de registro de los usuarios registrados después del 15 de enero de 2026, ordenados por fecha

```
1 SELECT nombre_usuario, fecha_registro
2 FROM USUARIO
3 WHERE fecha_registro > '2026-01-15'
4 ORDER BY fecha_registro;
```

Showing rows: 1 to 3 Page No: 1 of 1		
nombre_usuario [PK] character varying (50)	fecha_registro timestamp without time zone	
ThiagoBS	2026-01-15 09:45:00	
Usuario01	2026-01-18 16:20:00	
Usuario02	2026-01-20 18:05:00	

B1: (UPDATE) Cambia el estado de la incidencia con id=1 a 'Cerrada'

```
1 UPDATE INCIDENCIA
2 SET estado = 'Cerrada'
3 WHERE id = 1;
```

Showing rows: 1 to 1 Page No: 1 of 1		
id	estado	
1	Cerrada	

Query returned successfully in 70 msec.

B2: (UPDATE) Actualiza la valoración media del colaborador 'Usuario02' a 4.75

```
1 UPDATE COLABORADOR
2 SET valoracion_media = 4.75
3 WHERE usuario = 'Usuario02';
```

Showing rows: 1 to 1 Page No: 1 of 1		
usuario	valoracion_media	
Usuario02	4.75	

Query returned successfully in 52 msec.

B3: (DELETE) Elimina la especialización de 'Usuario02' en la categoría con id=5 (Seguridad)

```
1 DELETE FROM ESTAR_ESPECIALIZADO
2 WHERE usuario = 'Usuario02'
3 AND categoria = 5;
```

Showing rows: 1 to 1 Page No: 1 of 1		
usuario	categoria	
Usuario02	5	

Query returned successfully in 52 msec.

B4: (DELETE) Elimina el mensaje con id=5 del sistema

```
1 DELETE FROM MENSAJE
2 WHERE id = 5;
```

Data Output Messages Notifications

DELETE 1

Query returned successfully in 53 msec.

C1: Obtiene el id y título de cada incidencia junto con el nombre del usuario que la reportó, cruzando las tablas INCIDENCIA y REPORTADOR

```
1 SELECT INCIDENCIA.id, INCIDENCIA.titulo, REPORTADOR.usuario AS reportador
2 FROM INCIDENCIA, REPORTADOR
3 WHERE INCIDENCIA.reportador = REPORTADOR.usuario;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

id	titulo	reportador
1	Error al cargar la página	JesusBP
2	Problema con la base de datos	KhaledKN
3	Permisos incorrectos de usuario	ThiagoBS
4	Consulta lenta en el sistema	Usuario01
5	Validación de formularios insuficie...	Usuario02

C2: Obtiene cada colaborador junto con la descripción de la solución que ha aportado, cruzando las tablas COLABORADOR, RESOLVER y SOLUCION

```
1 SELECT COLABORADOR.usuario AS colaborador, SOLUCION.descripcion AS solucion
2 FROM COLABORADOR, RESOLVER, SOLUCION
3 WHERE COLABORADOR.usuario = RESOLVER.colaborador
4 AND RESOLVER.solucion = SOLUCION.id;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

colaborador	solucion
JesusBP	Reiniciar el servidor y limpiar la caché.
KhaledKN	Actualizar la base de datos a la versión más reciente.
ThiagoBS	Cambiar la configuración de permisos de usuario.
Usuario01	Optimizar la consulta SQL que genera el error.
Usuario02	Agregar validación de datos en el formulario de entra...

C3: Obtiene el id y título de cada incidencia junto con el nombre de la categoría a la que pertenece, cruzando las tablas INCIDENCIA, CLASIFICAR y CATEGORIA

```
1 SELECT INCIDENCIA.id AS incidencia, INCIDENCIA.titulo, CATEGORIA.nombre AS ca
2 FROM INCIDENCIA, CLASIFICAR, CATEGORIA
3 WHERE INCIDENCIA.id = CLASIFICAR.incendencia
4 AND CLASIFICAR.categoria = CATEGORIA.id;
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

incidencia	titulo	categoria
1	Error al cargar la página	Frontend
2	Problema con la base de datos	Base de datos
3	Permisos incorrectos de usuario	Seguridad
4	Consulta lenta en el sistema	Backend
5	Validación de formularios insuficie...	Redes

D1: Cuenta el número total de usuarios registrados en el sistema

PROGRAMA INCIDENCIAS

```
1 SELECT COUNT(*) AS total_usuarios
2 FROM USUARIO;
```

	total_usuarios	bigint
1	5	

D2: Calcula la valoración media global de todos los colaboradores

```
1 SELECT AVG(valoracion_medio) AS valoracion_promedio
2 FROM COLABORADOR;
```

	valoracion_promedio	numeric
1	4.4900000000000000	

D3: Obtiene el valor más alto entre todas las recompensas disponibles

```
1 SELECT MAX(valor) AS recompensa_mayor
2 FROM RECOMPENSA;
```

	recompensa_mayor	numeric
1	100.00	

E1: Muestra el número de incidencias agrupadas por zona geográfica

```
1 SELECT ZONA.nombre AS zona, COUNT(INCIDENCIA.id) AS total_incidencias
2 FROM INCIDENCIA, ZONA
3 WHERE INCIDENCIA.zona = ZONA.id
4 GROUP BY ZONA.nombre;
```

	zona	character varying (100)	totalIncidencias	bigint
1	Bilbao		1	
2	Barcelona		1	
3	Sevilla		1	
4	Madrid		1	
5	Valencia		1	

E2: Muestra la fecha del primer comentario realizado por cada usuario

```
1 SELECT USUARIO.nombre_usuario, MIN(COMENTARIO.fecha) AS primera_fecha_comenta
2 FROM USUARIO, COMENTARIO
3 WHERE USUARIO.nombre_usuario = COMENTARIO.usuario
4 GROUP BY USUARIO.nombre_usuario;
```

	nombre_usuario	[PK] character varying (50)	primera_fecha_comentario	timestamp without time zone
1	KhaledKN		2026-02-05 11:15:00	
2	Usuario01		2026-02-06 14:45:00	
3	Usuario02		2026-02-07 16:20:00	
4	ThiagoBS		2026-02-06 09:30:00	
5	JesusBP		2026-02-05 10:00:00	

F1: Obtiene los reportadores que han creado más incidencias que la media del sistema

```

1 SELECT usuario
2 FROM REPORTADOR
3 WHERE total_creadas > (
4     SELECT AVG(total_creadas)
5     FROM REPORTADOR
6 );

```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1

	usuario [PK] character varying (50)
1	JesusBP
2	ThiagoBS

F2: Obtiene el id y título de las incidencias cuya recompensa asociada supera el valor medio de todas las recompensas

```

1 SELECT INCIDENCIA.id, INCIDENCIA.titulo
2 FROM INCIDENCIA, RECOMPENSA
3 WHERE INCIDENCIA.id = RECOMPENSA.incidencia
4 AND RECOMPENSA.recompensa IN (
5     SELECT id
6     FROM RECOMPENSA
7     WHERE valor > (SELECT AVG(valor) FROM RECOMPENSA)
8 );

```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

	id [PK] numeric (10)	titulo character varying (200)
1	1	Error al cargar la página
2	2	Problema con la base de dat...

G1: Muestra los usuarios que tienen al menos 1 comentario, junto con el total de comentarios de cada uno

```

1 SELECT COMENTARIO.usuario, COUNT(*) AS total_comentarios
2 FROM COMENTARIO
3 GROUP BY COMENTARIO.usuario
4 HAVING COUNT(*) >= 1;

```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	usuario character varying (50)	total_comentarios bigint
1	KhaledKN	1
2	Usuario01	1
3	Usuario02	1
4	ThiagoBS	1
5	JesusBP	1

G2: Muestra las zonas que tienen al menos 1 incidencia registrada, junto con el total de incidencias

```

1 SELECT Z.nombre AS zona, COUNT(I.id) AS total_incidencias
2 FROM INCIDENCIA I, ZONA Z
3 WHERE I.zona = Z.id
4 GROUP BY Z.nombre
5 HAVING COUNT(I.id) >= 1;

```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	zona character varying (100)	total_incidencias bigint
1	Bilbao	1
2	Barcelona	1
3	Sevilla	1
4	Madrid	1
5	Valencia	1

H1: Cierra todas las incidencias cuya solución asociada ha sido marcada como aceptada

```

1  UPDATE INCIDENCIA
2  SET estado = 'Cerrada'
3  WHERE id IN (
4      SELECT R.incidencia
5      FROM RESOLVER R, SOLUCION S
6      WHERE R.solucion = S.id
7      AND S.es_aceptada = TRUE
8  );

```

Data Output Messages Notifications

UPDATE 3

Query returned successfully in 63 msec.

H2: Actualiza el campo total_resueltas de cada colaborador contando las entradas reales que tiene en la tabla RESOLVER

```

1  UPDATE COLABORADOR C
2  SET total_resueltas = (
3      SELECT COUNT(*)
4      FROM RESOLVER R
5      WHERE R.colaborador = C.usuario
6  );

```

Data Output Messages Notifications

UPDATE 5

Query returned successfully in 53 msec.

H3: Recalcula de nuevo el total de incidencias resueltas por cada colaborador a partir de los datos actuales de RESOLVER

```

123  UPDATE REPORTADOR R
124  SET total_creadas = (
125      SELECT COUNT(*)
126      FROM INCIDENCIA I
127      WHERE I.reportador = R.usuario
128  );

```

Data Output Messages Notifications

UPDATE 5

Query returned successfully in 97 msec.

VISTAS

V1: Une las tablas de incidencias, zonas y categorías devolviendo una fila por incidencia con su zona y todas sus categorías concatenadas en un único campo de texto separado por comas.

```
CREATE OR REPLACE VIEW vista_incidencias_con_zona_categoria AS
SELECT
    i.id,
    i.estado,
    i.titulo,
    i.descripcion,
    i.fecha_creacion,
    i.reportador,
    z.nombre AS nombre_zona,
    STRING_AGG(c.nombre, ', ') AS categorias
FROM INCIDENCIA i, ZONA z, CLASIFICAR cl, CATEGORIA c
WHERE i.zona = z.id
    AND cl.incidencia = i.id
    AND cl.categoria = c.id
GROUP BY i.id, i.estado, i.titulo, i.descripcion, i.fecha_creacion, i.reportador, z.nombre;
```

Permite obtener la información de una incidencia en una sola consulta simple, sin tener que relacionar tablas desde el código. El paquete DAO la usa directamente con un SELECT * para construir los objetos.

V2: Une las tablas COLABORADOR y USUARIO para mostrar un ranking de colaboradores ordenado de mayor a menor por número de incidencias resueltas, incluyendo su email y valoración media.

```
CREATE OR REPLACE VIEW vista_top_colaboradores AS
SELECT
    u.nombre_usuario AS usuario,
    u.email,
    c.total_resueltas AS incidencias_resueltas,
    c.valoracion_media
FROM COLABORADOR c, USUARIO u
WHERE c.usuario = u.nombre_usuario
ORDER BY c.total_resueltas DESC;
```

Permite mostrar fácilmente una tabla de clasificación de colaboradores, ya que los datos llegan directamente ordenados y listos para poner en la interfaz.

FUNCIONES

F1: Recibe el nombre de un usuario y devuelve el número total de incidencias que ha reportado contando sus registros en la tabla INCIDENCIA.

```

CREATE OR REPLACE FUNCTION fn_total_incidentes_reportador(p_usuario VARCHAR)
RETURNS NUMERIC AS $$
DECLARE
    v_total NUMERIC;
BEGIN
    SELECT COUNT(*) INTO v_total
    FROM INCIDENCIA
    WHERE reportador = p_usuario;

    RETURN v_total;
END;
$$ LANGUAGE plpgsql;

```

Permite obtener de forma rápida el contador de incidencias creadas por un usuario concreto, sirve para mostrar estadísticas en su perfil sin necesidad de construir la consulta desde el código Java.

F2: Recibe el nombre de una zona y devuelve el número de incidencias con estado Abierta que pertenecen a ella. Si no hay ninguna devuelve 0 gracias al COALESCE.

```

CREATE OR REPLACE FUNCTION fn_incidentes_abiertas_por_zona(p_zona VARCHAR)
RETURNS NUMERIC AS $$
DECLARE
    v_total NUMERIC;
BEGIN
    SELECT COUNT(*) INTO v_total
    FROM INCIDENCIA i, ZONA z
    WHERE z.id = i.zona
        AND z.nombre = p_zona
        AND i.estado = 'Abierta';

    RETURN COALESCE(v_total, 0);
END;
$$ LANGUAGE plpgsql;

```

Permite consultar la carga de incidencias pendientes por zona de forma directa, sirve para mostrar estadísticas o resúmenes en el panel de administración sin lógica adicional en el código Java.

F3: Crea una incidencia completa a partir de datos básicos: resuelve los IDs de zona y categoría por nombre, inserta la incidencia con estado Abierta y fecha actual, incrementa el contador total_creadas del reportador y la clasifica en la categoría indicada, todo en una sola operación.

```

CREATE OR REPLACE FUNCTION insertar_incidencia(
  p_titulo      VARCHAR,
  p_descripcion VARCHAR,
  p_reportador  VARCHAR,
  p_nombre_zona VARCHAR,
  p_nombre_cat  VARCHAR
) RETURNS VOID AS $$
DECLARE
  v_id_incidencia NUMERIC;
  v_id_zona       NUMERIC;
  v_id_categoria  NUMERIC;
BEGIN
  SELECT id INTO v_id_zona FROM ZONA WHERE nombre = p_nombre_zona;
  SELECT id INTO v_id_categoria FROM CATEGORIA WHERE nombre = p_nombre_cat;
  SELECT COALESCE(MAX(id), 0) + 1 INTO v_id_incidencia FROM INCIDENCIA;

  INSERT INTO INCIDENCIA (id, estado, titulo, descripcion, fecha_creacion, reportador, zona)
  VALUES (v_id_incidencia, 'Abierta', p_titulo, p_descripcion, CURRENT_DATE, p_reportador, v_id_zona);

  -- Suma +1 al contador de incidencias creadas del reportador
  UPDATE REPORTADOR
  SET total_creadas = total_creadas + 1
  WHERE usuario = p_reportador;

  INSERT INTO CLASIFICAR (incidencia, categoria)
  VALUES (v_id_incidencia, v_id_categoria);
END;
$$ LANGUAGE plpgsql;

```

Centraliza toda la lógica de creación de una incidencia en la base de datos, de modo que desde el código Java basta con una única llamada `SELECT insertar_incidencia(...)` pasando los nombres directamente, sin gestionar IDs ni múltiples inserts desde la capa DAO.

F4: Recorre todos los colaboradores con un cursor y recalcula su valoración media comparando el número de valoraciones recibidas con el total de incidencias resueltas. El resultado se escala sobre 5 y se limita a ese máximo con `LEAST`. Si no tiene valoraciones o resueltas asigna 0.

```

CREATE OR REPLACE FUNCTION fn_recalcular_valoraciones()
RETURNS VOID AS $$
DECLARE
    cur_colaboradores CURSOR FOR
        SELECT usuario FROM COLABORADOR;

    v_usuario COLABORADOR.usuario%TYPE;
    v_total NUMERIC;
    v_resueltas NUMERIC; -- ← declarada
    v_media NUMERIC;
BEGIN
    OPEN cur_colaboradores;

    LOOP
        FETCH cur_colaboradores INTO v_usuario;
        EXIT WHEN NOT FOUND;

        SELECT COUNT(*) INTO v_total
        FROM RESOLVER r, VALORAR v
        WHERE r.incidencia = v.incidencia
            AND r.solucion = v.solucion
            AND r.colaborador = v_usuario;

        SELECT total_resueltas INTO v_resueltas -- ← rellenada
        FROM COLABORADOR
        WHERE usuario = v_usuario;

        IF v_total > 0 AND v_resueltas > 0 THEN
            v_media := ROUND((v_total * 5.0) / v_resueltas, 2);
        ELSE
            v_media := 0.00;
        END IF;

        UPDATE COLABORADOR
        SET valoracion_media = LEAST(v_media, 5.00)
        WHERE usuario = v_usuario;

    END LOOP;

    CLOSE cur_colaboradores;
END;
$$ LANGUAGE plpgsql;

```

Permite actualizar de golpe las valoraciones de todos los colaboradores con una sola llamada, sirve para ejecutarla desde el panel de administración cuando se quiera refrescar las estadísticas globales sin calcular nada desde el código Java.

F5: Elimina un usuario y todos sus datos relacionados respetando el orden de las claves ajenas: primero borra sus registros en RESOLVER (como colaborador y como reportador), luego sus clasificaciones e incidencias, después sus roles en REPORTADOR y COLABORADOR, y finalmente el registro principal en USUARIO.

```

CREATE OR REPLACE FUNCTION eliminar_usuario(p_usuario VARCHAR)
RETURNS VOID AS $$
BEGIN

    DELETE FROM RESOLVER WHERE colaborador = p_usuario;

    DELETE FROM RESOLVER
    WHERE incidencia IN (
        SELECT id FROM INCIDENCIA WHERE reportador = p_usuario
    );

    DELETE FROM CLASIFICAR
    WHERE incidencia IN (
        SELECT id FROM INCIDENCIA WHERE reportador = p_usuario
    );

    DELETE FROM INCIDENCIA WHERE reportador = p_usuario;

    DELETE FROM REPORTADOR WHERE usuario = p_usuario;
    DELETE FROM COLABORADOR WHERE usuario = p_usuario;

    DELETE FROM USUARIO WHERE nombre_usuario = p_usuario;
END;
$$ LANGUAGE plpgsql;

```

Permite borrar un usuario completo sin preocuparse por el orden de las tablas ni por errores de la base de datos. Desde Java se hace una sola llamada y la función se encarga de todo.

TRIGGERS

T1: Antes de eliminar cualquier incidencia, el trigger comprueba si su estado es 'En progreso'. Si es así, lanza un error con el título de la incidencia e impide el borrado. Solo deja eliminar incidencias que estén en otro estado.

```
CREATE OR REPLACE FUNCTION fn_check_borrado_incidencia()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.estado = 'En progreso' THEN
        RAISE EXCEPTION
            'No se puede eliminar la incidencia "%" porque está En progreso. Ciérrala primero.',
            OLD.titulo;
    END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE TRIGGER trg_no_borrar_en_progreso
BEFORE DELETE ON INCIDENCIA
FOR EACH ROW
EXECUTE FUNCTION fn_check_borrado_incidencia();
```

Protege automáticamente los datos sin necesidad de añadir validaciones en Java. Si alguien intenta borrar una incidencia que está siendo atendida, la base de datos lo bloquea directamente y devuelve un mensaje claro indicando que debe cerrarse primero.

T2: Cada vez que se registra un usuario nuevo, el trigger lo inscribe automáticamente en las tablas COLABORADOR y REPORTADOR con los contadores a cero, sin necesidad de hacerlo manualmente.

```
CREATE OR REPLACE FUNCTION fn_auto_registrar_rolles()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO COLABORADOR (usuario, valoracion_media, total_resueltas)
    VALUES (NEW.nombre_usuario, 0.00, 0);

    INSERT INTO REPORTADOR (usuario, total_creadas)
    VALUES (NEW.nombre_usuario, 0);

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE TRIGGER trg_nuevo_usuario_rolles
AFTER INSERT ON USUARIO
FOR EACH ROW
EXECUTE FUNCTION fn_auto_registrar_rolles();
```

Garantiza que todo usuario registrado pueda tanto reportar incidencias como colaborar en resolverlas desde el primer momento. El registro de roles es transparente y no requiere ninguna lógica adicional en el código Java.

T3: Escucha cualquier cambio en la tabla RESOLVER y actualiza automáticamente el contador total_resueltas del colaborador: suma 1 al insertar una solución, resta 1 al eliminarla, y si se reasigna la solución a otro colaborador descuenta al anterior y suma al nuevo.

```
CREATE OR REPLACE FUNCTION fn_gestionar_total_resueltas()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        UPDATE COLABORADOR SET total_resueltas = total_resueltas + 1 WHERE usuario = NEW.colaborador;

    ELSIF TG_OP = 'DELETE' THEN
        UPDATE COLABORADOR SET total_resueltas = total_resueltas - 1 WHERE usuario = OLD.colaborador;

    ELSIF TG_OP = 'UPDATE' THEN
        IF OLD.colaborador IS DISTINCT FROM NEW.colaborador THEN
            UPDATE COLABORADOR SET total_resueltas = total_resueltas - 1 WHERE usuario = OLD.colaborador;
            UPDATE COLABORADOR SET total_resueltas = total_resueltas + 1 WHERE usuario = NEW.colaborador;
        END IF;
    END IF;

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE OR REPLACE TRIGGER trg_resolver_gestiona_contador
AFTER INSERT OR UPDATE OR DELETE ON RESOLVER
FOR EACH ROW
EXECUTE FUNCTION fn_gestionar_total_resueltas();
```

Mantiene el contador de incidencias resueltas siempre actualizado sin ninguna intervención desde Java. Cualquier operación sobre RESOLVER refleja el cambio al instante en el perfil del colaborador.

T4: Antes de registrar una solución en RESOLVER, comprueba si el colaborador es el mismo usuario que reportó la incidencia. Si es así, lanza un error y cancela la operación.

```

CREATE OR REPLACE FUNCTION fn_check_colaborador_no_es_reportador()
RETURNS TRIGGER AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM INCIDENCIA i
        WHERE i.id = NEW.incidencia
        AND i.reportador = NEW.colaborador
    ) THEN
        RAISE EXCEPTION
            'El colaborador "%" no puede resolver su propia incidencia (id: %).',
            NEW.colaborador, NEW.incidencia;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE OR REPLACE TRIGGER trg_colaborador_no_resuelve_su_incidencia
BEFORE INSERT ON RESOLVER
FOR EACH ROW
EXECUTE FUNCTION fn_check_colaborador_no_es_reportador();

```

Impide que un usuario se asigne como resolvidor de sus propias incidencias directamente desde la base de datos, sin depender de validaciones en Java. Garantiza la integridad del proceso de resolución en cualquier circunstancia.

INTEFAZ GRAFICA

Antes de comenzar con la explicación de la interfaz gráfica, es importante aclarar que los diagramas de clases y demás aspectos técnicos se encuentran detallados en la documentación general del proyecto. En esta sección se abordará únicamente la interfaz y su relación con la base de datos.

Ventana Inicio Sesión

Nombre de Usuario:

Contraseña:

Iniciar Sesion

[¿No tienes cuenta?](#)

Registrarse

Esta ventana permite acceder a la aplicación. La validación se realiza mediante una consulta a la base de datos que verifica el usuario y la contraseña; si ambos coinciden, se concede acceso a la ventana principal. Existen dos tipos de acceso: administrador (ADMIN) y usuario estándar.

Ventana Registrarse

Crear cuenta

Nombre de Usuario:

Correo:

Contraseña:

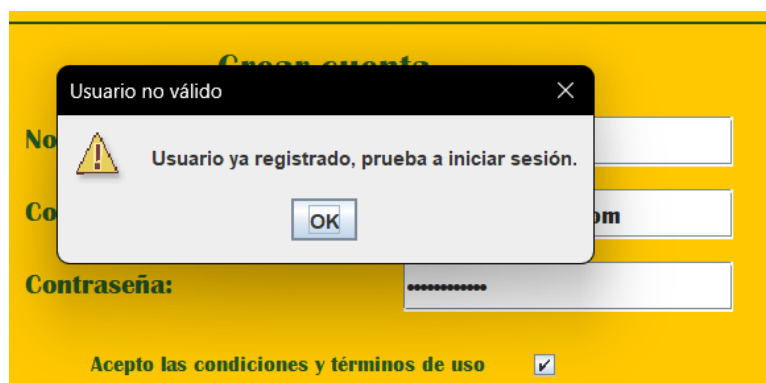
Acepto las condiciones y términos de uso ☐

Registrarse

Volver

Esta ventana permite crear un nuevo usuario en la base de datos para acceder al programa. Para ello, se realiza una inserción de los datos mediante una consulta INSERT.

Además, cuenta con una validación que realiza una consulta SELECT para comprobar si el nombre de usuario, el cual actúa como clave primaria (PK), ya existe en la base de datos. En caso de existir, se mostrará un aviso indicando que el usuario ya está registrado.



Ventana Admin

The screenshot shows the 'FIX IT! ADMIN' web application. It features a yellow header with the logo and a 'Opciones' menu. The main content area is divided into three columns, each with a table of data and a corresponding action button.

INCIDENCIAS: 10

ID	Estado	Título
1	Abierta	Wifi sin conexi...
2	Cerrada	Error al guardar ...
3	Abierta	Boton de acceso ...
4	Cerrada	Servicio de notifi...
5	Abierta	Permisos incorre...
6	Cerrada	Monitor del labora...
7	Abierta	Aplicacion de inci...
8	Cerrada	Listado de incide...
9	Abierta	Solicitud de ayud...
10	Cerrada	Manual de instala...

Ver **Eliminar**

USUARIOS: 10

Usuario	Email	Rol
JesusBP	jesusbp@alu.edu...	Admin
KhaledKN	khaledkn@alu.ed...	Usuario
ThiagoBS	thiagobs@alu.edu...	Usuario
AnaMG	anamg@alu.edu.g...	Usuario
CarlosRS	carlosrs@alu.edu...	Usuario
LuciaPR	luciapr@alu.edu.g...	Usuario
MartaLC	martalc@alu.edu...	Usuario
PabloGT	pablogt@alu.edu...	Usuario
NereaVF	nereavf@alu.edu...	Usuario
SergioDM	sergiodm@alu.ed...	Usuario

Ver **Eliminar**

MENSAJES RECIBIDOS: 10

Nombre	Asunto
JesusBP	Consulta sobre red
KhaledKN	Error en base de datos
ThiagoBS	Problema visual
AnaMG	Notificaciones
CarlosRS	Permisos
LuciaPR	Monitor averiado
MartaLC	Aplicacion bloqueada
PabloGT	Rendimiento
NereaVF	Impresora
SergioDM	Documentacion

Leer Mensaje

En esta ventana se muestra un panel principal donde se puede visualizar en tiempo real la cantidad de incidencias, usuarios y mensajes recibidos. Esta información se obtiene mediante consultas SELECT realizadas sobre las tablas correspondientes de la base de datos.

- **Incidencias:**

Se muestran las incidencias registradas en el sistema con la información más relevante. El administrador puede acceder a cada incidencia para ampliar su información. Además, tiene la posibilidad de eliminar incidencias. Para esta funcionalidad se utiliza la vista [V1](#).



The screenshot shows the 'FIXIT! ADMIN' window. It contains a form with the following fields: 'Estado' (Cerrada), 'Titulo' (Manual de instalacion incompleto), 'Reportador' (JesusBP), 'Zona' (Castellon), 'Fecha' (2026-02-10), and 'Categorias' (Documentacion). Below these is a 'Descripcion' field with the text 'La guia no explica como configurar la conexion con PostgreSQL.'. At the bottom, there are two buttons: 'Marcar como abierta' (green) and 'Marcar como cerrada' (yellow).

Al acceder a la información de una incidencia, el administrador (ADMIN) puede abrirla o cerrarla según su estado y necesidad de gestión.

- **Usuarios:**

Se presenta la lista de usuarios registrados en el sistema. El administrador puede consultar la información detallada de cada usuario y, si es necesario, eliminarlo. Para la eliminación de usuarios se utiliza la función [F5](#).



The screenshot shows the 'FIXIT! ADMIN' window displaying user details for 'AnaMG'. The details are as follows:

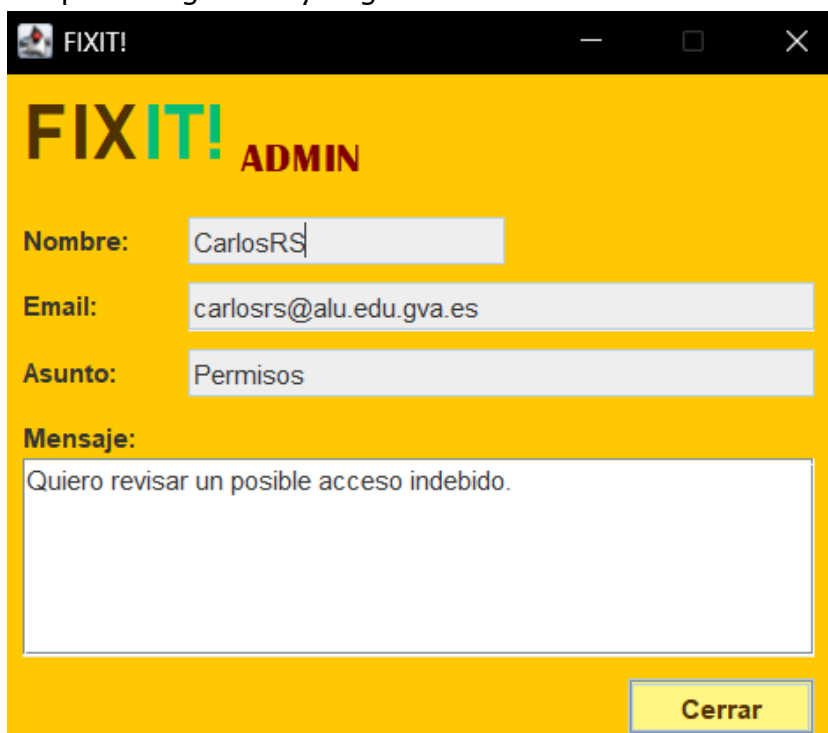
Email:	anamg@alu.edu.gva.es
Fecha de registro:	2026-01-13
Incidencias creadas:	6
Incidencias resueltas:	8

At the bottom, there is a green button labeled 'Volver'.

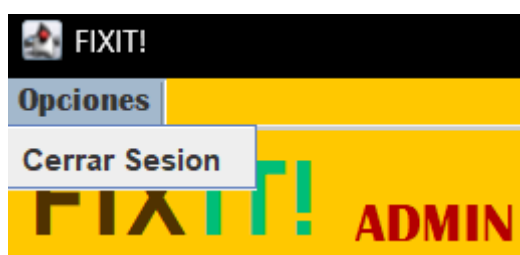
Al pulsar el botón “Ver”, se abre el perfil del usuario seleccionado. Los datos del perfil se cargan mediante consultas SELECT a las tablas Usuario, Colaborador y Reportador.

- **Mensajes:**

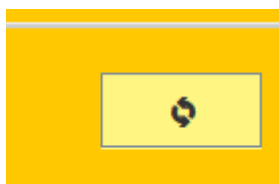
Se visualizan los mensajes recibidos, mostrando los datos más importantes de cada uno para su gestión y seguimiento.



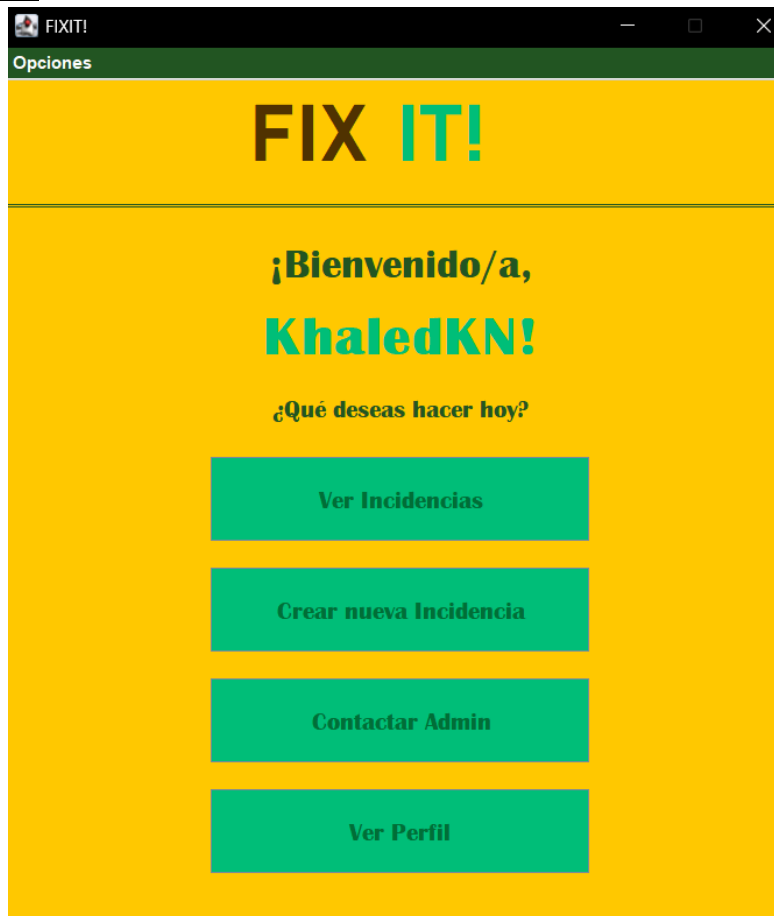
Al pulsar “Leer mensaje”, se cargarán todos los campos del mensaje, mostrando el usuario que lo envió junto con su contenido.



La ventana dispone de un menú que incluye la opción de cerrar sesión, permitiendo al usuario iniciar sesión con otra cuenta si lo desea.



Por último, la ventana incluye un botón para recargar las listas, con el fin de mantener los datos actualizados en tiempo real.

Ventana Usuario

La ventana muestra un mensaje de bienvenida en el que se ponen a disposición cuatro opciones: ver incidencias, crear nueva incidencia, contactar con el administrador y ver perfil. Cada una de estas opciones abre una nueva ventana.

Al igual que en la ventana del administrador, esta pantalla también dispone de un menú con la opción de cerrar sesión.

Ventana Ver Incidencias


ID	Título	Estado	Zona	Categoría	Reportador
3	Boton de acceso desali.	Abierta	Barcelona	Frontend	AnaMG
5	Permisos incorrectos e.	Abierta	Bilbao	Seguridad	LuciaPR
7	Aplicacion de incidencia	Abierta	Murcia	Software	PabloGT
9	Solicitud de ayuda con i.	Abierta	Malaga	Soporte	SergioDM

En esta ventana se muestran las incidencias de la base de datos cuyo estado es “Abierto” y cuyo reportador no corresponde al propio usuario. Para ello se utiliza la vista V1, aplicando desde Java las condiciones mencionadas.



Título: Boton de acceso desalineado

Zona: Barcelona

Categoría: Frontend

Descripción:
El boton de iniciar sesion aparece fuera de lugar en pantallas pequenas.

Tu solución:

Es posible seleccionar una incidencia y, al pulsar la opción “Ofrecerse a resolver”, se abre una nueva ventana que permite enviar un mensaje con la posible solución al problema del reportador.

Al enviar la solución, la incidencia actualiza su estado a “cerrada” mediante una sentencia UPDATE. Además, gracias al trigger T3, se actualiza automáticamente el contador de incidencias resueltas.

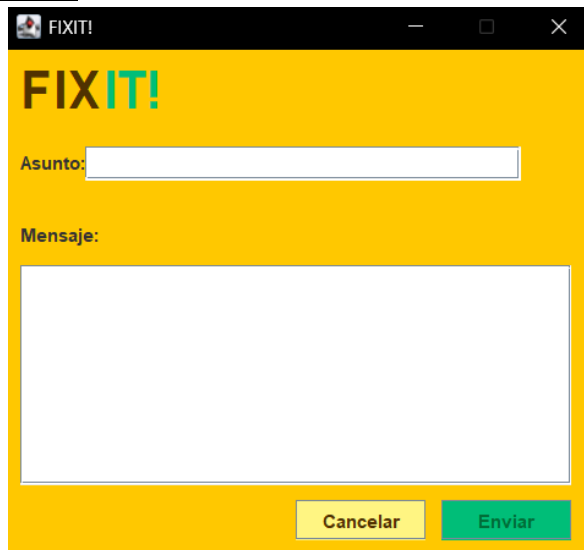
Ventana Crear Nueva Incidencia



The screenshot shows a window titled "Nueva Incidencia" from the "FIX IT!" application. The window has a yellow background. At the top, there is a header with the "FIX IT!" logo and the title "Nueva Incidencia". Below the header, there are two input fields: "Titulo" (a single-line text box) and "Descripción" (a multi-line text area). Under these fields, there are two list boxes. The "Zona" list box contains the following items: Valencia, Madrid, Barcelona, Sevilla, Bilbao, and Alicante. The "Categoria" list box contains the following items: Redes, Base de datos, Frontend, Backend, Seguridad, and Hardware. At the bottom of the window, there are two buttons: "Cancelar" (a yellow button) and "Agregar Incidencia" (a green button).

Esta ventana se utiliza para reportar incidencias. En ella se muestran las categorías y zonas obtenidas de la base de datos mediante una lista.

Al pulsar “Agregar incidencia”, se invoca la función [F3](#), que se encarga de realizar el INSERT en la base de datos y de incrementar en +1 el contador de incidencias creadas.

Ventana Contactar Admin

FIXIT!

FIXIT!

Asunto:

Mensaje:

Cancelar Enviar

Esta ventana permite generar un registro en la tabla Contacto. Posteriormente, este mensaje se visualizará en la lista de "Mensajes" del panel de administración.

El usuario únicamente debe introducir el asunto y el mensaje, ya que desde Java se obtiene automáticamente el usuario de la sesión y sus datos asociados.

Ventana Ver Perfil

FIXIT! Mi Perfil

KhaledKN

Email:	khaledkn@alu.edu.gva.es
Fecha de registro:	2026-01-11
Incidencias creadas:	8
Incidencias resueltas:	10

Volver

En esta ventana se muestran los datos del usuario, de forma similar a la ventana de visualización de usuarios del panel de administración.